CSA47 – Deep Learning

CO1 – ASSESSMENT 1: ASSIGNMENT-1

CO1: *Mathematical foundations of deep learning – linear algebra, probability and information theory, numerical computation, machine learning basics, capacity, overfitting and underfitting, estimation, bias and variance, Bayesian statistics, stochastic gradient descent, challenges motivating deep learning.*

Assessment Title: Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System (Total Marks - 50)

Nature of Assessment: Individual Duration: 1 Week Bloom's Level: K4 – Analyse
Submission Date: 12.10.26

Name: DHAANUSH RAJAN

Reg No: 192421162

Scenario — "AgriVision" Crop-Disease Screening

AgriVision is an agri-tech start-up that helps farmers detect leaf diseases from smartphone photographs. Before the company invests in a deep neural network, its Chief Data Scientist wants a rigorous analysis of the mathematical and machine learning foundations of the problem.

You will work with a public leaf-disease image dataset of your choice (for example, the tomato subset of PlantVillage, with Healthy, Early Blight and Late Blight classes). Images may be resized to 64 × 64 grayscale, giving 4,096 raw pixel features per image.

The dataset may be used at increasing scales:

1,000 → 5,000 → 10,000 → full dataset images

You are required to examine how the data is represented, break down the sources of error and uncertainty in classical machine learning models, differentiate their optimisation and generalisation behaviour, and determine, with evidence, whether deep learning is justified. Every preprocessing step must be documented and examined for its effect on the results.

Design Challenge:

AgriVision now needs to support 38 crop–disease classes, add images whose labels are about 20% noisy, and run the classifier on a low-cost smartphone within 50 ms and 5 MB.

Analyse how these changes affect your findings and determine whether your conclusion should be retained, modified or redesigned.

TASK 1 – Analyse the Problem Structure and Data Representation [K4] 4 Marks

Break down AgriVision into its learning-task components and examine how the choice of data representation influences the problem.

Discuss:

- The learning task, input, output and performance measure, and how each constrains the solution
- How the dataset, one image, a mini-batch of 32 images and the labels are organised as scalars, vectors, matrices or tensors, and why each shape matters
- How the L^1, L^2 and max (L^∞) norms of the same feature vector differ, and which norm suits which analysis purpose
- How each preprocessing step (resizing, normalisation, flattening, data split) changes the information available to the model

You are expected to **examine the consequences of each representation choice**, not merely list the shapes.

Evidence Expected

A decomposition of the task into its components and an analysis of how representation and preprocessing affect the problem.

TASK 2 – Examine the Structure of the Feature Space using Linear Algebra [K4]5 Marks

Use eigendecomposition and/or singular value decomposition to examine how information is distributed across the 4,096-dimensional image representation.

Discuss:

- How the eigenvalues or singular values are distributed, and what this reveals about redundancy among pixels
- The relationship between eigenvalues, singular values and explained variance
- How the reconstruction error changes as the number of components k varies, and where the trade-off point lies
- What the leading principal components represent when visualised as images
- How the cost of decomposition grows with the dataset size

Show at least one decomposition **worked by hand on a small 2 × 2 or 3 × 3 sub-problem** to support your analysis.

Evidence Expected

An analysis of the spectrum, an interpretation of the leading components and a justified compression trade-off point.

TASK 3 – Analyse Uncertainty using Probability and Information Theory [K4] 4 Marks

Examine the uncertainty in AgriVision's labels and predictions using probability and information theory.

Discuss:

- How the class imbalance is reflected in the entropy of the class distribution
- How the cross-entropy and KL divergence between the true and predicted distributions differ across your models, and what the differences reveal
- Why KL divergence is asymmetric, illustrated with your own numbers, and what each direction measures
- Why minimising cross-entropy and minimising KL divergence lead to the same classifier

You are expected to **relate every quantity to the behaviour of the classifier**, not merely compute it.

Evidence Expected

A comparative analysis of entropy, cross-entropy and KL divergence that explains model behaviour.

TASK 4 – Diagnose Numerical Computation Issues [K4] 3 Marks

Investigate where and why numerical failures occur when computing class probabilities and gradients.

Discuss:

- The conditions under which a naive softmax overflows or underflows (for example z = [1000, 999, 995])
- Why subtracting max(z) and the log-sum-exp trick remove the failure without changing the result
- How the condition number of the feature covariance matrix explains slow or unstable gradient-based training

Evidence Expected

A demonstrated failure traced to its cause, with an analysis of why the correction works.

TASK 5 – Differentiate Baseline Learning Strategies [K4] 4 Marks

Implement **at least three meaningfully different learning algorithms** on raw and PCA-compressed features and differentiate their behaviour.

The approaches must represent **distinct learning strategies** (for example: a linear parametric model, a non-parametric model and a kernel or tree-based model). Two settings of the same algorithm **will not be considered as two different approaches**.

Discuss:

- How the hypothesis space and inductive bias of each algorithm differ
- Which characteristics of 4,096-dimensional image data favour or hinder each algorithm
- How compression affects each algorithm differently, and why

Evidence Expected

Three distinct baselines with a reasoned analysis of why their performance differs.

TASK 6 – Analyse Capacity, Overfitting and Underfitting [K4] **4 Marks**

Examine how model capacity drives training and validation error for your baselines.

Discuss:

- How varying capacity (polynomial degree, k in k-NN, tree depth, regularisation strength) changes training and validation error
- Where the underfitting and overfitting regions lie, and what causes each
- How the generalisation gap changes with capacity
- Why selecting hyperparameters on the test set biases the reported error, compared with a validation set or k-fold cross-validation

Do not simply present curves. You are expected to **explain the causes behind each region**.

Evidence Expected

Capacity–error curves with each regime identified and its cause explained.

TASK 7 – Decompose Error into Bias and Variance [K4] **4 Marks**

Break down the error of your models into its components and examine the effect of prior knowledge on estimation.

Discuss:

- A bootstrap experiment (at least 10 replicas) separating bias² and variance for a low- and a high-capacity model
- How closely the measured error matches bias² + variance + noise, and why any gap exists
- How the MLE and MAP estimates of a class proportion differ under a Beta prior of your choice

- How the influence of the prior changes as the sample size grows

Evidence Expected

An experimental bias–variance decomposition and a comparative analysis of MLE and MAP estimates.

TASK 8 – Analyse the Behaviour of Gradient-Based Optimisation [K4] 4 Marks

Implement softmax regression **from scratch** and analyse how different optimisation strategies behave on it.

Discuss:

- The structure of the loss gradient and the terms that drive each update
- How batch gradient descent, stochastic gradient descent and mini-batch SGD differ in convergence path, noise and cost per epoch
- How at least three learning rates change stability and speed, and why
- How the observed behaviour relates to the conditioning identified in Task 4

You must show the **derivation of the gradient** to support your analysis.

Evidence Expected

A derived gradient and loss curves with an analysis of the differences between strategies and learning rates.

TASK 9 – Organise and Examine Experimental Performance [K4] 3 Marks

Organise the experimental results for all models across the dataset scales 1,000 → 5,000 → 10,000 → full, and examine the patterns they reveal.

Examine measures such as:

- Accuracy, precision, recall, F1-score and the confusion matrix
- Which classes are confused with each other, and why
- Training time, inference time, parameter count and memory footprint
- How each measure changes with dataset size and with compression

Use **tables, graphs and performance plots** that make the patterns and trade-offs visible.

Evidence Expected

Systematically organised results with the key patterns and trade-offs identified.

TASK 10 – Compare Theoretical Expectations with Experimental Behaviour [K4]3 Marks

Examine the agreement between your mathematical analysis and the observed behaviour.

Discuss:

- Whether explained variance accounts for the accuracy retained after compression
- Whether optimisation speed agrees with the condition number and learning-rate analysis
- Whether the bias–variance results agree with the capacity curves
- The causes of any disagreement

Evidence Expected

A reasoned analysis of where theory and experiment agree or differ, and why.

TASK 11 – Analyse whether Deep Learning is Justified [K4] 4 Marks

Using your findings, analyse whether the limitations of classical machine learning on AgriVision's data warrant a deep network.

Discuss:

- Which limitations of the best baseline remain after tuning
- How the curse of dimensionality, local constancy and smoothness, and manifold structure manifest in your data
- What evidence indicates the need for a learned hierarchical representation
- Trade-offs in data, compute and interpretability

Your conclusion must be **traced to evidence from your own analysis**. Stating that deep learning is more accurate is not sufficient.

Evidence Expected

An evidence-based analysis of the need for deep learning, linked to specific findings.

DAY-4 DESIGN CHALLENGE

TASK 12 – Analyse the Impact of Changed Requirements 8 Marks

Changed Requirement

AgriVision now needs to classify **38 crop–disease classes**. Newly collected images have about **20% noisy labels**, and the model must run on a **low-cost smartphone within 50 ms per image and 5 MB of memory**.

Re-evaluate your original design in response to these changed operating conditions. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- Which elements of the original problem have changed
- How the larger label space affects entropy, cross-entropy loss and baseline accuracy
- How label noise affects bias, variance and the estimated error
- Which original assumptions about data and compute no longer hold

Evidence Expected

A clear breakdown of how each change affects the original analysis.

B. Examination of the Original Conclusion – 2 Marks

Discuss:

- Whether your best model fits within the 50 ms and 5 MB limits, using your measurements
- Which parts of its behaviour are most sensitive to 38 classes and noisy labels
- Which alternative approach is less affected, and why

Evidence Expected

A measured examination of the original conclusion under the new conditions.

C. Analysis of Adaptation Options – 2 Marks

Examine at least two options for responding to the changes, such as:

- Changing the PCA dimension or regularisation strength
- Using a noise-robust loss or label smoothing
- Compressing or simplifying the model

For each option, analyse its effect on capacity, error and computational cost.

Evidence Expected

A comparative analysis of adaptation options with their effects on capacity, error and cost.

D. Evidence-Based Conclusion – 2 Marks

State whether you retain, modify or redesign your conclusion, and trace it to:

- Mathematical analysis (entropy, bias–variance, conditioning)
- Experimental results
- Inference-time and memory measurements
- The trade-offs identified in Part C

Your conclusion must follow logically from your analysis.

Evidence Expected

A conclusion whose every element is traced to the analysis in Parts A–C.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Problem and Data Representation Analysis	4
Task 2 – Feature-Space Analysis (Eigen/SVD)	5
Task 3 – Uncertainty Analysis	4
Task 4 – Numerical Stability Diagnosis	3
Task 5 – Comparative Analysis of Baselines	4
Task 6 – Capacity and Generalisation Analysis	4
Task 7 – Bias–Variance and Estimation Analysis	4
Task 8 – Optimisation Behaviour Analysis	4
Task 9 – Experimental Performance Analysis	3
Task 10 – Theory vs Experiment	3
Task 11 – Analysis of the Need for Deep Learning	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

ANSWER

Task 1 – Analyse the Problem Structure and Data Representation (4 marks)

Learning-task components

Component	AgriVision definition	Constraint on the solution
Task	Supervised, single-label, three-class image classification	Each image has one target label; a classifier must separate three visual categories.
Input	Smartphone or dataset image of a tomato leaf	Image quality, lighting, background and resolution influence the signal.
Output	Probability distribution over Healthy, Early Blight and Late Blight	Probabilities permit confidence analysis; the largest probability gives the predicted class.
Training signal	Human-provided class labels	Incorrect or inconsistent labels create a noisy target.
Performance	Accuracy plus macro precision, recall and F1; cross-entropy; confusion matrix	Macro metrics prevent a large class from hiding poor performance on a smaller class.
Deployment	Single-image inference on a phone	Latency, memory, energy and robustness constrain model selection.

Accuracy is the fraction of correct predictions. Precision measures how many predictions of a class are correct; recall measures how many examples of that class are found; F1 is the harmonic mean of precision and recall. Macro averages calculate each class metric independently and then average them, giving each class equal weight.

Accuracy = correct predictions / total predictions

F1 = 2 × Precision × Recall / (Precision + Recall)

Shapes and why they matter

Object	Shape	Interpretation and consequence
One grayscale image	64 × 64 matrix	Rows and columns preserve the two-dimensional spatial layout.
Flattened image	Vector x ∈ R^4096	Required by standard tabular ML models, but explicit neighbourhood structure is lost.
Dataset of N images	X ∈ R^(N × 4096)	Rows are examples; columns are pixel features. Memory grows linearly with N × 4096.
Mini-batch of 32	X_b ∈ R^(32 × 4096)	Allows vectorised matrix operations and reduces update noise compared with one-example SGD.
Three-class labels	y ∈ {0,1,2}^N	One integer per example is sufficient for many libraries.
One-hot labels	Y ∈ {0,1}^(N × 3)	Each row has one 1; convenient for cross-entropy and softmax gradients.
Class probabilities	P ∈ [0,1]^(N × 3)	Each row sums to 1 and expresses the model's predicted distribution.

L^1, L^2 and $L\infty$ norms

For the same vector x = [3, −4, 2], the norms are:

$\mathbf{||x||_1 = |3| + |-4| + |2| = 9}$

$\mathbf{||x||_2 = \sqrt{(3^2 + (-4)^2 + 2^2)} = \sqrt{29} \approx 5.385}$

$\mathbf{||x||_\infty = \max(|3|, |-4|, |2|) = 4}$

L^1 measures total absolute magnitude and is associated with sparse penalties when used in regularisation. L^2 measures Euclidean magnitude and is common in distance calculations and

ridge-style regularisation. L∞ measures the largest absolute coordinate and can be useful for worst-coordinate deviations. Norm values are scale-dependent; raw pixel norms should not be compared across differently sized or differently normalised images without accounting for that change.

Preprocessing effects

- Resizing to 64 × 64 reduces computation and storage but may erase tiny lesions, edges or texture patterns. Interpolation creates new pixel values rather than recovering lost detail.
- Grayscale reduces three RGB channels to one intensity channel. It saves two-thirds of the channel storage but removes colour cues that may distinguish chlorosis, necrosis or other symptoms.
- Normalising by 255 maps integer intensity values to [0, 1], improving numerical scale. It does not itself standardise each pixel to zero mean and unit variance.
- Flattening changes a 2-D image into a 1-D vector. A standard classifier can use the values, but it does not automatically know that nearby pixels form a lesion.
- A stratified train/validation/test split preserves approximate class ratios. The test set must remain untouched until final evaluation; hyperparameter selection belongs on validation data or inside cross-validation.

Task 1 conclusion: representation is a modelling decision, not a neutral file conversion. It trades spatial and colour information against speed, memory and compatibility with classical algorithms.

Task 2 – Examine the Feature Space Using Linear Algebra (5 marks)

PCA and the covariance matrix

Let X_train be the training matrix with N rows and 4,096 columns. Subtract the training-set mean from each feature to obtain X_c. The sample covariance matrix is $C = X_c^T X_c/(N-1)$, a 4,096 × 4,096 matrix. Eigendecomposition gives $C\, v_i = \lambda_i v_i$. The eigenvectors define principal directions and the eigenvalues describe variance along those directions.

$$\mathbf{C = V \Lambda V^T}$$

Alternatively, singular value decomposition is $X_c = U \Sigma V^T$. For the sample covariance matrix, $\lambda_i = \sigma_i^2/(N-1)$. The explained-variance ratio of component i is λ_i divided by the sum of all eigenvalues. A steeply decreasing spectrum indicates that many dimensions carry relatively little variance; a flatter spectrum indicates that more components are needed to retain variation. High variance is not identical to high disease-discriminative information.

Hand-worked 2 × 2 eigendecomposition (illustrative)

Consider the symmetric covariance-like matrix $A = [[2, 1], [1, 2]]$. Its characteristic equation is $\det(A - \lambda I) = (2-\lambda)^2 - 1 = 0$, so $\lambda = 3$ or $\lambda = 1$. For $\lambda_1 = 3$, an eigenvector is $[1, 1]^T$; after unit normalisation it is $[1/\sqrt{2}, 1/\sqrt{2}]^T$. For $\lambda_2 = 1$, an eigenvector is $[1, -1]^T$; after normalisation it is $[1/\sqrt{2}, -1/\sqrt{2}]^T$.

Explained variance of PC1 = 3 / (3 + 1) = 0.75 = 75%

Explained variance of PC2 = 1 / (3 + 1) = 0.25 = 25%

If only PC1 is retained, the discarded variance is 1 out of total variance 4, or 25%. This is a small hand example only; it is not an experimental result from the leaf dataset.

Reconstruction error and component choice

For k retained components, reconstruct each centred image using the projection onto the first k principal directions. The average squared reconstruction error should generally decrease as k increases. A practical choice is near an elbow in the reconstruction-error curve, but the elbow must be checked against validation classification metrics and latency. The script writes pca_spectrum.csv, pca_reconstruction.csv, pca_cumulative_variance.png and pca_components.png. Inspect the leading component images: they often show broad lighting, leaf silhouette or common spatial patterns, but their interpretation must be based on the generated plots.

k	Expected mathematical trend	What to report after running
2 / 5	Strong compression; usually larger reconstruction error	Measured reconstruction MSE and retained variance from pca_reconstruction.csv.
10 / 20	More structure retained; still compact	Whether the variance curve begins to flatten.
40 / 80 / 100 or more	Lower reconstruction error; higher feature cost	Validation macro-F1 and inference time as well as variance.

Computational cost

A dense covariance matrix has $4{,}096^2 = 16{,}777{,}216$ entries. At float64, the matrix alone occupies about 128 MiB, excluding intermediate arrays. Forming covariance scales approximately with $O(Nd^2)$, where $d = 4{,}096$; a full dense eigendecomposition scales

approximately with $O(d^3)$. Direct SVD of an N × d matrix also has substantial cost. Randomised or truncated SVD/PCA can reduce work when only a modest number of components is needed. For small N relative to d, dual or randomized methods may be preferable.

Task 2 conclusion: choose k from the measured spectrum, reconstruction error and downstream validation performance together. Do not claim a specific k is optimal before running the experiment.

Task 3 – Analyse Uncertainty Using Probability and Information Theory (4 marks)

Class entropy

$$\mathbf{H(Y) = -\Sigma_i\, p_i \log_2(p_i)}$$

Entropy is measured in bits when the logarithm base is 2. A balanced three-class distribution has entropy $\log_2(3) \approx 1.585$ bits. If classes are imbalanced, entropy is lower than this maximum. The actual class counts should be read from the dataset inventory. Class imbalance can cause a model to favour the majority class, so accuracy should be interpreted alongside macro recall and the confusion matrix.

Cross-entropy and KL divergence

$$\mathbf{H(p, q) = -\Sigma_i\, p_i \log(q_i)}$$

$$\mathbf{D_KL(p \,||\, q) = \Sigma_i\, p_i \log(p_i / q_i) = H(p, q) - H(p)}$$

Cross-entropy scores a predicted distribution q against the target distribution p. KL divergence measures the information lost when q is used to approximate p. For one-hot labels, cross-entropy penalises the negative log probability assigned to the correct class. Lower test cross-entropy indicates better probability assignment, but it should be considered with calibration, class-wise errors and accuracy.

KL asymmetry with numbers (illustrative)

Let p = [0.8, 0.2] and q = [0.6, 0.4]. Using natural logarithms:

$$\mathbf{D_KL(p \,||\, q) = 0.8 \ln(0.8/0.6) + 0.2 \ln(0.2/0.4) \approx 0.0915 \text{ nats}}$$

$$\mathbf{D_KL(q \,||\, p) = 0.6 \ln(0.6/0.8) + 0.4 \ln(0.4/0.2) \approx 0.1046 \text{ nats}}$$

The directions differ because the weights in the sum come from the first distribution. D_KL(p || q) evaluates q as an approximation to p; reversing the distributions changes the penalty. KL divergence is non-negative but is not a distance metric because it is asymmetric and does not satisfy the triangle inequality.

Why cross-entropy and KL can select the same classifier

For a fixed target distribution p, H(p) does not depend on the model q. Since H(p, q) = H(p) + D_KL(p || q), minimising cross-entropy over q is equivalent to minimising forward KL divergence. In supervised classification with fixed labels, the target entropy is constant across candidate models. Report both the formula and the empirical test cross-entropy produced by the script; do not invent model values.

Task 4 – Diagnose Numerical Computation Issues (3 marks)

Softmax overflow and stable computation

softmax(z_i) = exp(z_i) / Σ_j exp(z_j)

For logits z = [1000, 999, 995], direct exponentiation attempts to compute exp(1000), which exceeds the representable range of ordinary floating-point numbers. The numerator and denominator can become infinity, producing inf/inf and NaN. Very negative logits can underflow toward zero, potentially creating zero probabilities and undefined log probabilities.

softmax(z) = softmax(z − max(z))

Subtracting the same constant from every logit leaves the ratio unchanged because the common factor exp(c) cancels from numerator and denominator. For this example, the stable logits are [0, −1, −5], whose exponentials are finite. The stable probabilities are approximately [0.7275, 0.2676, 0.0049] (illustrative calculation). The experiment runner records naive and stable outputs in numerical_softmax.txt.

Conditioning and optimisation

The condition number κ(C) = λ_max/λ_min for a positive-definite covariance matrix describes the ratio between the largest and smallest curvature scales. A large condition number means the loss surface can be steep in some directions and flat in others. Gradient descent may zigzag, require a small learning rate, or converge slowly. If a feature has zero variance, C is singular and the condition number is infinite unless a regularisation or pseudo-inverse convention is used. In practice, compute a condition number on a manageable centred/scaled matrix or use nonzero singular values; document the method.

The covariance matrix for 4,096 pixel features can be highly correlated because adjacent pixels and broad leaf regions vary together. Standardisation and PCA can improve numerical conditioning, but fitting transformations only on training data is essential.

conclusion

Use max-subtracted softmax and a stable log-sum-exp calculation. Treat non-finite loss/gradients as a failure signal. Connect observed optimisation curves to the feature scaling and conditioning diagnostics rather than attributing every slow run to the algorithm alone.

Task 5 – Differentiate Baseline Learning Strategies (4 marks)

Algorithm	Hypothesis space / inductive bias	Strengths and limitations for 4,096 pixels
Regularised multinomial logistic regression	Linear decision boundaries in the feature space; regularisation limits coefficient magnitude.	Fast inference and compact parameter matrix. May miss nonlinear combinations of lesion texture and shape.
k-nearest neighbours (k-NN)	Non-parametric local prediction based on distances to stored examples.	Flexible local boundaries; inference and memory grow with training-set size. Distances become less informative in high dimensions.
Random forest	Ensemble of decision trees; partitions the feature space with nonlinear rules.	Captures nonlinear feature interactions and needs limited scaling. Many trees can consume memory; axis-aligned splits may be inefficient for image patterns.

Raw features are 4,096-dimensional and contain spatially correlated pixels. PCA features are lower-dimensional linear combinations of pixels, which can reduce distance-computation cost and noise, but PCA is unsupervised: directions with high variance are not necessarily the directions most useful for separating disease classes. The script compares all three algorithms on raw and 100-component PCA features. A fair comparison uses the same fixed test set and reports both predictive metrics and resource use.

Expected qualitative hypothesis—not a measured result: k-NN may be sensitive to distance concentration and dataset size; logistic regression may generalise well when class separation is approximately linear; random forests may capture some nonlinearities but can be larger. Confirm or reject these hypotheses using the CSV results.

PCA versus raw features

- For logistic regression, PCA may reduce redundant predictors and speed optimisation, but discarded low-variance directions can still carry class information.
- For k-NN, compression reduces the number of dimensions in each distance calculation and may make neighbours more meaningful; it can also distort neighbourhood structure.
- For random forest, PCA changes feature semantics from individual pixel intensities to mixed component scores. It may reduce dimensionality but can remove sparse local cues that tree splits could otherwise use.

Task 5 conclusion: no algorithm is automatically best. Select using validation performance, macro-F1, cross-entropy, inference latency and serialized model size.

Task 6 – Analyse Capacity, Overfitting and Underfitting (4 marks)

Capacity is the range of decision functions a model can represent. A low-capacity model may not capture disease patterns and underfits; a high-capacity model can fit training-specific details and overfit. Training error usually does not increase as a model becomes more flexible, while validation error can first decrease and then increase.

Capacity setting	**Low-capacity region**	**High-capacity region / trade-off**
Random forest depth	Shallow trees may miss interactions and underfit.	Deep trees can memorise idiosyncrasies; ensembles and leaf constraints help control variance.
k in k-NN	Large k averages over broad neighbourhoods and can underfit.	Small k creates irregular local boundaries and may be sensitive to noise; k=1 can have near-zero training error.
Logistic regression C	Strong regularisation (small C) shrinks coefficients and may underfit.	Weak regularisation (large C) permits larger coefficients and can increase variance.
PCA components k	Too few components discard potentially useful information.	Many components retain noise/redundancy and increase cost; PCA itself does not guarantee overfitting but changes the feature space.

The experiment runner creates train-versus-validation error curves for forest depth and k-NN neighbour count. Label underfitting where both errors are high; label overfitting where training error is much lower than validation error. The generalisation gap is validation error minus training error. These diagnoses should be made from the actual plotted results.
The test set must not be used to choose depth, k, C or PCA dimension. Repeated test-set tuning leaks information from the test labels into model selection and makes the final score optimistically biased. Use a validation split or k-fold cross-validation on the training partition, then evaluate the chosen configuration once on the held-out test set.

The dataset should be separated into three parts:

Training set: Used to fit model parameters.

Validation set: Used to choose hyperparameters, such as \(k\), tree depth and regularisation strength.

Test set: Used for the final, unbiased-as-practical evaluation of the selected model.

If the test set is repeatedly used to choose hyperparameters, decisions gradually become tailored to its examples. The reported test error can then underestimate the model's true error on new images.

When data is limited, k-fold cross-validation provides an alternative: divide the training data into \(k\) folds, train on \(k-1\) folds and validate on the remaining fold, repeating until every fold has served as validation data. Choose hyperparameters from the cross-validation results, then evaluate the selected model on the untouched test set.

Task 7 – Decompose Error into Bias and Variance (4 marks)

Bootstrap experiment

The experiment runner draws ten bootstrap resamples of the training set and trains a low-capacity regularised logistic model and a high-capacity random forest on each resample. It evaluates their predicted class-probability vectors on a fixed test subset. Since this is classification, the script uses a Brier-score-style squared probability error as a practical proxy:

Brier = mean$_i$ Σ_c ($\hat{p}_i$c − 1[y_i=c])²

The mean prediction across bootstrap replicas estimates the stable component. The squared difference between that mean and the one-hot target is reported as a bias² proxy; the variance across replica probabilities is reported as a variance proxy. The mean observed squared error is also reported. This is an empirical proxy, not a perfect universal decomposition of 0–1

classification error. Finite replicas, finite test data, covariance between terms and irreducible label uncertainty can make measured error differ from bias² + variance + noise.

Typical expectation: a highly constrained model may show higher bias and lower variance; a flexible model may lower bias but show higher variance. This is a hypothesis to test, not a guaranteed ranking for every dataset.

MLE and MAP with a Beta prior

For a binary event such as whether a randomly selected training example is Healthy, let n be the number of training examples and h the number labelled Healthy. The maximum-likelihood estimate is p_MLE = h/n. With a Beta(α, β) prior, the posterior is Beta(h+α, n−h+β). Its posterior mean is (h+α)/(n+α+β). When α,β > 1, the posterior mode (MAP) is (h+α−1)/(n+α+β−2).

Illustrative arithmetic: if h=60 out of n=100 and the prior is Beta(2,2), then MLE = 0.60 and MAP mode = (60+1)/(100+2) ≈ 0.598. These are illustrative counts, not PlantVillage measurements. The actual script computes MLE and MAP mode from the training split and writes results/mle_map.csv.

As n grows, fixed prior pseudo-counts have less influence relative to the observed data, so MLE and MAP generally become closer. The prior can stabilise estimates with small samples, but a poorly chosen prior can introduce bias.

conclusion

Use bootstrap distributions rather than one training run to examine stability. Report the exact replica count, test subset size and chosen error proxy. Explain any mismatch between observed error and the approximate bias–variance sum.

9. Task 8 – Analyse Gradient-Based Optimisation (4 marks)

9.1 Softmax regression and gradient derivation

For a mini-batch with feature matrix X ∈ R^(m×d), weights W ∈ R^(d×C), bias b ∈ R^C and one-hot labels Y ∈ {0,1}^(m×C), logits are Z = XW + b. Apply row-wise softmax to obtain P. The mean cross-entropy is:

$$\mathbf{L(W,b) = -(1/m)\ \Sigma_i\ \Sigma_c\ Y_ic\ \log\ P_ic}$$

For one example, the derivative of softmax cross-entropy with respect to logit z_c is p_c − y_c. Applying the chain rule to Z = XW+b gives:

$$\mathbf{\partial L/\partial W = X^T(P - Y)/m}$$

$$\mathbf{\partial L/\partial b = \Sigma_i(P_i - Y_i)/m}$$

$$\mathbf{W \leftarrow W - \eta\ \partial L/\partial W\ ;\quad b \leftarrow b - \eta\ \partial L/\partial b}$$

The update is driven by the difference between predicted and target probabilities, weighted by the input features. The implementation uses a max-subtracted softmax for numerical stability. It uses PCA features for the from-scratch optimisation comparison to keep the computation manageable; the PCA fit is done on the training subset.

9.2 Batch GD, SGD and mini-batch SGD

Strategy	**Update data**	**Convergence and cost**
Batch gradient descent	All training examples per update	Smooth gradient direction but one update costs a full pass; each epoch has one update.
Stochastic gradient descent	One example per update	Frequent, noisy updates; low work per update but high variance and many updates per epoch.
Mini-batch SGD	A batch such as 32 examples	Vectorised computation with moderate noise; often a practical balance between throughput and update stability.

The runner compares learning rates 0.0001, 0.001 and 0.01 for all three strategies. A learning rate that is too small produces slow loss reduction; one that is too large can oscillate or diverge. The safe range depends on feature scaling and loss curvature. Compare training and validation cross-entropy curves in results/softmax_*.png and inspect softmax_optimisation.csv.

Conditioning from Task 4 affects the path: correlated pixel features can create elongated loss contours. Standardisation and PCA can reduce some redundancy, but PCA component scaling must be understood because variance differs across components. If curves become non-finite, check the learning rate, stable softmax and input scaling.

Task 8 conclusion

Select the optimiser and learning rate from observed convergence speed, stability and validation loss, not training loss alone. Mention wall-clock cost as well as number of updates.

Task 9 – Organise and Examine Experimental Performance (3 marks)

The experiment runner produces a structured comparison across available training scales and six model/feature combinations. For each run, it records accuracy, macro precision, macro recall, macro-F1, cross-entropy, training time, median and 95th-percentile single-image inference latency, and compressed serialized model size. Confusion matrices are saved separately.

Evidence file	**What to examine**
results/model_scale_metrics.csv	Performance and resource trends from 1,000 → 5,000 → 10,000 → full training samples.
results/cm_*.csv	Which class pairs are confused; look for Early/Late Blight confusion and minority-class recall.
results/pca_spectrum.csv	Per-component and cumulative explained variance.
results/pca_reconstruction.csv	Reconstruction MSE and variance retained at several component counts.
results/capacity_errors.csv	Training and validation error across capacity settings.
results/bias_variance.csv	Bootstrap bias² proxy, variance proxy and measured Brier-style error.
results/softmax_optimisation.csv	Training and validation loss for optimiser/learning-rate combinations.
results/label_noise_stress_test.csv	Test performance after simulated 20% training-label corruption.
results/run_metadata.json	Machine, software versions, class list and dataset counts.

Use the following table to enter real values after running the script. Do not leave blank cells disguised as measured outcomes; if a run fails, record “not measured” and explain why.

Scale	**Model**	**Accuracy**	**Macro-F1**	**Train time (s)**	**Median latency (ms)**	**Model (MB)**
1,000	Logistic raw / PCA	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV

1,000	k-NN raw / PCA	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV
1,000	Random forest raw / PCA	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV
5,000	All six combinations	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV
10,000	All six combinations	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV
Full	All six combinations	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV	Fill from CSV

Interpretation checklist: Does adding data improve validation/test metrics or mainly increase cost? Does PCA speed inference enough to justify any accuracy loss? Are accuracy and macro-F1 telling different stories? Which disease pair dominates the off-diagonal confusion matrix? Are the measured latency and model size below deployment limits? Results are hardware-specific and should include the computer used.

Task 10 – Compare Theory with Experimental Behaviour (3 marks)

Theoretical expectation	**Evidence to compare**	**How to explain disagreement**
High explained variance may retain broad image structure.	Compare PCA variance and reconstruction MSE with macro-F1/accuracy.	Variance is unsupervised; low-variance disease cues may be discriminative, while high variance may reflect lighting/background.
Better conditioning can improve gradient optimisation.	Compare optimiser curves with raw/scaled/PCA features and learning rates.	Runtime noise, learning-rate choice, stochastic updates and model implementation also affect convergence.
Capacity affects bias and variance.	Compare capacity curves and bootstrap proxies.	Finite sample size, class overlap, noisy labels and the chosen proxy can change the expected pattern.

More training data often reduces estimation uncertainty.	Compare scale metrics on the same held-out test set.	Performance can plateau, class distribution can shift, or additional samples can be redundant or noisy.

This task should be completed from the generated results. Avoid asserting that theory and experiment agree without naming a particular plot, CSV row or measured quantity. Any mismatch is useful evidence when it is traced to representation, finite sample effects, label noise, model assumptions or measurement variation.

Task 11 – Analyse Whether Deep Learning Is Justified (4 marks)

Evidence to assess

- Residual baseline errors: examine the confusion matrices and class-wise precision/recall after reasonable tuning. If similar disease symptoms remain confused, inspect whether the features need spatial texture and hierarchical representations.
- Curse of dimensionality: in 4,096 dimensions, examples can be sparse and pairwise distances can concentrate, weakening local methods such as k-NN. More data and dimensionality reduction may help, but neither guarantees good separation.
- Local constancy and smoothness: k-NN assumes nearby feature vectors tend to have similar labels. Illumination changes, leaf orientation and background variation can make pixel-space neighbours biologically dissimilar; visually similar leaves can also carry different disease labels.
- Manifold structure: natural images lie on a structured subset of the full pixel space. Changes in pose, scale, lighting and background produce complex variations that may be difficult to represent with flat pixel vectors.
- Hierarchical representation: convolutional or other vision networks can learn low-level edges and textures, then combine them into lesions and broader patterns. Evidence for the need is persistent baseline error linked to such patterns, not merely the general popularity of deep learning.

12.2 Trade-offs

A deep network may improve feature learning and accuracy when enough diverse labelled data and compute are available. It also increases training cost, may require careful calibration and explanation, and can fail on field images unlike the controlled dataset. Classical models are easier to audit and can be smaller or faster, especially a linear classifier after

dimensionality reduction. A fair decision compares all methods on the same split and under the same preprocessing.

Decision rule: deep learning is justified only if the best tuned classical baseline has meaningful, operationally important errors and a suitable compact network demonstrates a reproducible improvement that survives the latency and memory tests. If the baseline already meets quality and resource targets, retain it until stronger evidence supports replacement.

Important limitation: this assignment's selected three-class tomato subset cannot establish the performance of a 38-class crop–disease system. It is a baseline study. The final design challenge needs the wider 38-class dataset and a separate deployment benchmark.

Task 12 – Day-4 Design Challenge: Changed Requirements (8 marks)

A. Impact Analysis (2 marks)

The changed requirements expand the output space from three tomato classes to 38 crop–disease classes, introduce approximately 20% noisy labels in newly collected images, and impose hard deployment limits of 50 ms per image and 5 MB model memory. The three-class data assumption, clean-label assumption and unconstrained desktop-compute assumption are no longer sufficient.

For a balanced distribution, maximum class entropy increases from $\log_2(3) \approx 1.585$ bits to $\log_2(38) \approx 5.248$ bits. Actual entropy depends on class proportions and may be lower. A uniform random guess has expected accuracy 1/3 for three classes and $1/38 \approx 2.63\%$ for 38 classes; this is a theoretical baseline, not a model result. More classes create more possible confusions and can increase cross-entropy if probability mass is assigned to the wrong class.

Random label corruption increases irreducible ambiguity and can make the model fit incorrect targets. It can raise estimation variance, distort class boundaries and bias measured performance if the test labels are also noisy. Keep a clean, independently verified test set if possible; apply simulated or observed label noise to training only for controlled comparisons.

B. Examine the Original Conclusion (2 marks)

Use the measured median and p95 inference latency and compressed model size from model_scale_metrics.csv as the first screen. The acceptance limits are latency $\leq$ 50 ms per image and model size $\leq$ 5 MB. The script's serialized joblib size is a useful comparison but is not automatically identical to a production mobile runtime footprint; include interpreter/runtime overhead and measure on the target phone before claiming compliance.

A linear model may have a small parameter footprint, while k-NN stores training examples and can have expensive inference; random forests can grow with the number and depth of trees. Increasing the class count increases the output layer and may increase confusion. Noise

sensitivity depends on the loss, regularisation and model capacity. The less affected alternative cannot be declared without measurements; test a regularised linear model, a noise-robust training variant and a compact vision model on the 38-class data.

C. Analyse Adaptation Options (2 marks)

Adaptation	**Expected effect on capacity/error**	**Compute and evidence needed**
Reduce PCA components	Lowers feature dimension and may regularise; too few components can remove disease cues.	Usually lowers memory and distance/model cost; choose k by validation macro-F1 and latency.
Increase regularisation	Shrinks coefficients or constrains model complexity; may reduce variance under label noise but can underfit.	Often little inference overhead; tune only on training/validation data.
Noise-robust loss or label smoothing	Can reduce overconfident fitting to corrupted labels; too much smoothing can blur class distinctions.	Small extra compute for smoothing; robust losses may add tuning complexity. Compare clean validation cross-entropy and macro-F1.
Simplify/compress the model	Quantisation, pruning or a smaller architecture can reduce capacity and size; accuracy may decrease.	Measure target-device latency, RAM/flash footprint and accuracy after compression.
Curate labels and use class-aware sampling	Improves target quality and minority representation; does not replace robust modelling.	Requires review effort and a verified validation set; track class counts before/after cleaning.

D. Evidence-Based Conclusion (2 marks)

Provisional recommendation: MODIFY the original design rather than automatically retain or fully redesign it. Keep the three-class classical study as a baseline, but do not treat it as proof of a 38-class deployable solution. Re-run the pipeline on the full 38-class dataset, measure

real 20% noise stress tests, evaluate a noise-robust alternative and benchmark on the intended low-cost phone. Choose the final model only if it meets the 50 ms and 5 MB limits while preserving acceptable macro-F1 and class-wise recall.

This recommendation follows from: (1) the theoretical rise in maximum label entropy from three to 38 classes; (2) the possibility that label noise increases estimation error; (3) the relationship between dimensionality/conditioning and optimisation; and (4) the fact that latency and memory limits are empirical properties that must be measured on the target hardware. Replace this provisional conclusion with the measured result after the extended experiments.

Design-challenge evidence table to complete:

Required evidence	**Result to enter**	**Decision implication**
38-class macro-F1 and per-class recall	Run on all 38 classes; enter measured values	Identifies weak crop–disease classes.
20% training-label noise comparison	Clean-label baseline vs noisy-label training	Quantifies robustness loss.
Target-phone median and p95 latency	Measure with production runtime	Must remain ≤ 50 ms per image.
Production model footprint	Measure deployed artifact and runtime memory	Model artifact target ≤ 5 MB; clarify what is included.
PCA/regularisation/compression ablation	Enter validation and resource metrics	Select the best quality–cost trade-off.